Experience-Weighted Attraction

General Principles

Social learning models often aim to disentangle asocial learning (learning from personal experience) from social learning (learning from others). The **Experience-Weighted Attraction (EWA)** model Barrett, McElreath, and Perry (2017) provides a robust framework for this. In the context of the Panama capuchin monkey study Barrett, McElreath, and Perry (2017), the model tracks how individuals update their "attractions" to different foraging techniques based on their own success and the social cues they observe from others.

The core idea is that individuals have a set of attractions $A_{i,j,t}$ for each technique j. These attractions are updated over time, and the probability of choosing a technique is a mixture of asocial preferences (derived from attractions) and social influence.

In this documentation, we present two primary variations of the EWA model adapted from Barrett, McElreath, and Perry (2017):

1. **Social Learning (EWA)**: A streamlined version (Time Frequency Pay-off model) focusing on the interplay between individual experience (asocial learning) and social frequency biased by payoffs. It uses a reduced parameter set (4 varying effects) to identify the core drivers of behavioral diffusion.
2. **Social Learning (EWA-ILV)**: A comprehensive hierarchical model (Experience-Weighted Attraction with Individual Level Covariates) that evaluates multiple social biases (e.g., kinship, prestige, age similarity) and explicitly models how an individual's age influences their learning rate (ϕ) and reliance on social cues (γ).

---

1. Social Learning (EWA)

The "EWA" model is a basic version that includes asocial learning and pay-off biased social learning. It evaluates four varying effects: learning rate, social weight, conformity, and pay-off bias.

Example

> **Note**
>
> - **Hierarchical Structure**: Intercepts for learning rates and social influence vary by individual (`id`). Barrett, McElreath, and Perry (2017) use a Cholesky-factored multivariate normal prior to account for correlations between these effects.
> - **Dtype Sensitivity**: Ensure all data arrays are `float64` to match JAX's 64-bit backend requirements for numerical stability.

Python

```python
from BayesForge import bf
import jax.numpy as jnp
import jax

m = bf(platform='cpu')

# Data: capuchin foraging choices (panama_ewa) ------------------------------
import pandas as pd
d = pd.read_csv(m.load.panama_ewa(only_path=True))
K = 7  # foraging techniques

def cue(prefix, scale=False):
    """N x K matrix of the columns prefix1..prefix7, optionally divided by its maximum."""
    x = d[[f"{prefix}{k}" for k in range(1, K + 1)]].values
    return jnp.array(x / x.max() if scale else x)

y = cue("y")                                    # personal yield of each technique
y_prev = jnp.zeros_like(y).at[1:].set(y[:-1])  # yields of the previous row

m.data_on_model = dict(
    K=K,
    J=int(d["mono_index"].nunique()),            # number of monkeys
    tech=jnp.array(d["tech_index"].values - 1),  # chosen technique, 0-indexed
    id=jnp.array(d["mono_index"].values - 1),    # monkey, 0-indexed
    bout=jnp.array(d["forg_bout"].values),       # 1 = first bout (attractions reset)
    y_prev=y_prev,
    s=cue("s"),                                  # frequency cue
    ps=cue("ps", scale=True),                    # payoff cue
)
```

```
def model(K, J, tech, id, bout, y_prev, s, ps):
    # Priors
    lam   = m.dist.exponential(name="lambda", rate=1.0)
    mu    = m.dist.normal(name="mu", loc=jnp.zeros(4), scale=1.0)
    sigma = m.dist.exponential(name="sigma", rate=jnp.full(4, 3.0))
    L_Rho = m.dist.lkj_cholesky(name="L_Rho", dimension=4, concentration=3.0)
    z     = m.dist.normal(name="z", loc=jnp.zeros((4, J)), scale=1.0)

    # Varying effects
    L_scaled = L_Rho * sigma[:, None]
    a_id     = (L_scaled @ z).T

    # Parameters per individual
    phi_arr   = jax.nn.sigmoid(mu[0] + a_id[id, 0])      # Learning rate
    gamma_arr = jax.nn.sigmoid(mu[1] + a_id[id, 1])      # Social weight
    fconf_arr = jnp.exp(mu[2] + a_id[id, 2])             # Conformity
    Bpay_arr  = mu[3] + a_id[id, 3]                      # Pay-off bias

    # Scan carry: Attractions
    def step_fn(carry, x):
        AC = carry
        (phi_i, gamma_i, fconf_i, Bpay_i, tech_i, id_i, bout_i, prev_y_i, s_i, ps_i) = x

        # Update attractions (EWA)
        ac_new = jnp.where(bout_i == 1, jnp.zeros(K), (1.0 - phi_i) * AC[id_i] + phi_i * prev

        # Asocial choice probability
        prob_individual = jax.nn.softmax(lam * ac_new)

        # Social choice probability (Pay-off bias)
        lin_mod = jnp.concatenate([jnp.ones(1), jnp.exp(Bpay_i * ps_i[1:])]) * jnp.power(s_i
        safe_denom = jnp.where(jnp.sum(lin_mod) > 0, jnp.sum(lin_mod), 1.0)
        prob_social = lin_mod / safe_denom

        # Mixture probability
        prob_mix = (1.0 - gamma_i) * prob_individual + gamma_i * prob_social
        has_social_info = jnp.logical_and(bout_i > 1, jnp.sum(s_i) > 0)
        prob_final = jnp.where(has_social_info, prob_mix, prob_individual)

        return AC.at[id_i].set(ac_new), prob_final

    AC_init = jnp.zeros((J, K))
```

```
    xs = (phi_arr, gamma_arr, fconf_arr, Bpay_arr, tech, id, bout, y_prev, s, ps)
    _, probs_matrix = jax.lax.scan(step_fn, AC_init, xs)

    m.dist.categorical(name="obs", probs=probs_matrix, obs=tech)

m.fit(model, seed=0)
m.summary()
```

R

```
library(BayesForge)
m <- importBF(platform = "cpu")
jnp <- reticulate::import("jax.numpy")
jax <- reticulate::import("jax")
py <- reticulate::import_builtins()

# Data: capuchin foraging choices (panama_ewa) ------------------------------
d <- read.csv(m$load$panama_ewa(only_path = TRUE))
K <- 7L  # foraging techniques

# N x K matrix of the columns prefix1..prefix7, optionally divided by its maximum
cue <- function(prefix, scale = FALSE) {
  x <- as.matrix(d[, paste0(prefix, 1:K)])
  jnp$array(if (scale) x / max(x) else x)
}

y <- as.matrix(d[, paste0("y", 1:K)])
y_prev <- rbind(0, y[-nrow(y), ])  # yields of the previous row

m$data_on_model <- list(
  K = K,
  J = length(unique(d$mono_index)),               # number of monkeys
  tech = jnp$array(as.integer(d$tech_index - 1)),  # chosen technique, 0-indexed
  id = jnp$array(as.integer(d$mono_index - 1)),    # monkey, 0-indexed
  bout = jnp$array(as.integer(d$forg_bout)),       # 1 = first bout (attractions reset)
  y_prev = jnp$array(y_prev),
  s = cue("s"),                                    # frequency cue
  ps = cue("ps", scale = TRUE)                     # payoff cue
)

# x[1:] of a JAX vector (all techniques but the first)
```

```r
rest <- function(x) x$`__getitem__`(py$slice(1L, NULL))

model <- function(K, J, tech, id, bout, y_prev, s, ps) {
  # Priors
  lam   <- m$dist$exponential(name = "lambda", rate = 1.0)
  mu    <- m$dist$normal(name = "mu", loc = jnp$zeros(4L), scale = 1.0)
  sigma <- m$dist$exponential(name = "sigma", rate = jnp$full(4L, 3.0))
  L_Rho <- m$dist$lkj_cholesky(name = "L_Rho", dimension = 4L, concentration = 3.0)
  z     <- m$dist$normal(name = "z", loc = jnp$zeros(reticulate::tuple(4L, J)), scale = 1.0)

  # Varying effects
  L_scaled <- L_Rho * jnp$expand_dims(sigma, -1L)
  a_id     <- jnp$transpose(jnp$matmul(L_scaled, z))

  # Parameters per individual
  phi_arr   <- jax$nn$sigmoid(mu[0L] + a_id[id, 0L])  # Learning rate
  gamma_arr <- jax$nn$sigmoid(mu[1L] + a_id[id, 1L])  # Social weight
  fconf_arr <- jnp$exp(mu[2L] + a_id[id, 2L])         # Conformity
  Bpay_arr  <- mu[3L] + a_id[id, 3L]                  # Pay-off bias

  # Scan carry: Attractions
  step_fn <- function(carry, x) {
    AC <- carry
    phi_i <- x[[1]]; gamma_i <- x[[2]]; fconf_i <- x[[3]]; Bpay_i <- x[[4]]
    tech_i <- x[[5]]; id_i <- x[[6]]; bout_i <- x[[7]]; prev_y_i <- x[[8]]
    s_i <- x[[9]]; ps_i <- x[[10]]

    # Update attractions (EWA)
    ac_new <- jnp$where(bout_i == 1L, jnp$zeros(K), (1.0 - phi_i) * AC[id_i] + phi_i * prev_

    # Asocial choice probability
    prob_individual <- jax$nn$softmax(lam * ac_new)

    # Social choice probability (Pay-off bias)
    lin_mod <- jnp$concatenate(list(jnp$ones(1L), jnp$exp(Bpay_i * rest(ps_i)))) * jnp$power
    safe_denom <- jnp$where(jnp$sum(lin_mod) > 0, jnp$sum(lin_mod), 1.0)
    prob_social <- lin_mod / safe_denom

    # Mixture probability
    prob_mix <- (1.0 - gamma_i) * prob_individual + gamma_i * prob_social
    has_social_info <- jnp$logical_and(bout_i > 1L, jnp$sum(s_i) > 0)
    prob_final <- jnp$where(has_social_info, prob_mix, prob_individual)
```

```
    reticulate::tuple(AC$at[id_i]$set(ac_new), prob_final)
  }

  AC_init <- jnp$zeros(reticulate::tuple(J, K))
  xs <- reticulate::tuple(phi_arr, gamma_arr, fconf_arr, Bpay_arr, tech, id, bout, y_prev, s
  probs_matrix <- jax$lax$scan(step_fn, AC_init, xs)[[2]]

  m$dist$categorical(name = "obs", probs = probs_matrix, obs = tech)
}

m$fit(model, seed = 0L)
m$summary()
```

Julia

```
using PyBayesForge
using PythonCall
using CSV, DataFrames
m = importBF(platform="cpu")

# Data: capuchin foraging choices (panama_ewa) ------------------------------
d = CSV.read(pyconvert(String, m.load.panama_ewa(only_path=true).py), DataFrame)
K = 7  # foraging techniques

# N x K matrix of the columns prefix1..prefix7, optionally divided by its maximum
function cue(prefix; scale=false)
    x = Matrix{Float64}(d[:, ["$(prefix)$(k)" for k in 1:K]])
    jnp.array(scale ? x ./ maximum(x) : x)
end

y = Matrix{Float64}(d[:, ["y$(k)" for k in 1:K]])
y_prev = vcat(zeros(1, K), y[1:end-1, :])  # yields of the previous row

m.data_on_model = pydict(
    K = K,
    J = length(unique(d.mono_index)),          # number of monkeys
    tech = jnp.array(d.tech_index .- 1),       # chosen technique, 0-indexed
    id = jnp.array(d.mono_index .- 1),         # monkey, 0-indexed
    bout = jnp.array(d.forg_bout),             # 1 = first bout (attractions reset)
    y_prev = jnp.array(y_prev),
```

```
    s = cue("s"),                                   # frequency cue
    ps = cue("ps", scale=true),                     # payoff cue
)

# x[1:] of a JAX vector (all techniques but the first); Python indexing is 0-based
rest(x) = x[pybuiltins.slice(1, nothing)]

@BF function model(K, J, tech, id, bout, y_prev, s, ps)
    # Priors
    lam   = m.dist.exponential(name="lambda", rate=1.0)
    mu    = m.dist.normal(name="mu", loc=jnp.zeros(4), scale=1.0)
    sigma = m.dist.exponential(name="sigma", rate=jnp.full(4, 3.0))
    L_Rho = m.dist.lkj_cholesky(name="L_Rho", dimension=4, concentration=3.0)
    z     = m.dist.normal(name="z", loc=jnp.zeros((4, J)), scale=1.0)

    # Varying effects
    L_scaled = L_Rho * jnp.expand_dims(sigma, -1)
    a_id     = jnp.transpose(jnp.matmul(L_scaled, z))

    # Parameters per individual
    phi_arr   = jax.nn.sigmoid(mu[0] + a_id[id, 0])  # Learning rate
    gamma_arr = jax.nn.sigmoid(mu[1] + a_id[id, 1])  # Social weight
    fconf_arr = jnp.exp(mu[2] + a_id[id, 2])         # Conformity
    Bpay_arr  = mu[3] + a_id[id, 3]                  # Pay-off bias

    # Scan carry: Attractions
    function step_fn(AC, x)
        AC = Py(AC)
        phi_i, gamma_i, fconf_i, Bpay_i, tech_i, id_i, bout_i, prev_y_i, s_i, ps_i = map(Py,

        # Update attractions (EWA)
        ac_new = jnp.where(bout_i == 1, jnp.zeros(K), (1.0 - phi_i) * AC[id_i] + phi_i * prev

        # Asocial choice probability
        prob_individual = jax.nn.softmax(lam * ac_new)

        # Social choice probability (Pay-off bias)
        lin_mod = jnp.concatenate(pylist([jnp.ones(1), jnp.exp(Bpay_i * rest(ps_i))])) * jnp
        safe_denom = jnp.where(jnp.sum(lin_mod) > 0, jnp.sum(lin_mod), 1.0)
        prob_social = lin_mod / safe_denom

        # Mixture probability
```

```
        prob_mix = (1.0 - gamma_i) * prob_individual + gamma_i * prob_social
        has_social_info = jnp.logical_and(bout_i > 1, jnp.sum(s_i) > 0)
        prob_final = jnp.where(has_social_info, prob_mix, prob_individual)

        return (AC.at[id_i].set(ac_new), prob_final)
    end

    AC_init = jnp.zeros((J, K))
    xs = (phi_arr, gamma_arr, fconf_arr, Bpay_arr, tech, id, bout, y_prev, s, ps)
    probs_matrix = jax.lax.scan(step_fn, AC_init, xs)[1]

    m.dist.categorical(name="obs", probs=probs_matrix, obs=tech)
end

m.fit(model, seed=0)
m.summary()
```

Mathematical Details

The "Experience-Weighted Attraction" (EWA) model links individual learning rates and social frequency to behavioral choice. Throughout this section, we use a consistent three-part index: i denotes the individual, $k \in \{1, \ldots, K\}$ denotes the technique, and t denotes the observation (time step).

1. Mixture Likelihood

At each observation, the probability of choosing a single technique from K options is expressed through a *Categorical* likelihood:

$$Y_{[i,t]} \sim \text{Categorical}(\theta_{[i,t]})$$

where:

- $Y_{[i,t]}$ is the technique chosen by individual i at time step t.
- $\theta_{[i,t]}$ is a probability vector of length K giving the probability of each technique for individual i at time step t.

Unlike a standard Categorical model where θ is fixed for a group of observations, here $\theta_{[i,t]}$ is a **different vector at every time step** for every individual, because the attraction scores and social cues update sequentially.

The probability that individual i chooses technique k at time t decomposes as:

$$\theta_{[i,k,t]} = (1 - \gamma_i)\, I_{[i,k,t]} + \gamma_i\, S_{[i,k,t]}$$

$$\gamma_i = \text{sigmoid}(\mu_2 + a_{id}[i, 2])$$

Where:

- $I_{[i,k,t]}$ is the asocial choice probability: derived from individual i's personal experience with technique k.
- $S_{[i,k,t]}$ is the social choice probability: derived from observing conspecifics use technique k.
- $\gamma_i \in [0, 1]$ is individual i's social learning weight: at 0 the individual relies entirely on personal experience; at 1 entirely on social information. It is derived from the hierarchical prior (see §2):

2. Asocial Component — $I_{[i,k,t]}$

The asocial probability is a softmax over accumulated attraction scores — it converts raw attraction values into a proper probability distribution over techniques:

$$I_{[i,k,t]} = \frac{\exp(\lambda \cdot A_{[i,k,t]})}{\sum_{k'=1}^{K} \exp(\lambda \cdot A_{[i,k',t]})}$$

$$\lambda \sim \text{Exponential}(1)$$

The numerator scores technique k by the individual's scaled attraction to it. The denominator sums this over all K techniques ($k' = 1, \ldots, K$), ensuring the probabilities sum to 1 and that each technique is evaluated relative to all alternatives.

Where:

- $A_{[i,k,t]}$ is the attraction score of individual i for technique k at time t. It is a running exponentially weighted average of the personal payoffs individual i has obtained from technique k across past bouts. Larger values indicate a stronger learned payoff expectation.
- $A_{[i,k',t]}$ in the denominator ranges over the attraction scores for all K techniques, making choice proportional to each option's scaled attraction relative to the full set.
- $\lambda > 0$ is the multinomial sensitivity (inverse temperature): higher values make choice more deterministic toward the highest-attraction option; $\lambda \to 0$ yields random choice. The Exponential(1) prior keeps λ positive and places most mass near zero, expressing a prior expectation that choice is moderately stochastic. Larger values are possible but require evidence from the data.

Attraction scores update between foraging bouts via the EWA rule:

$$A_{[i,k,t+1]} = (1 - \phi_i)\, A_{[i,k,t]} + \phi_i\, p_{[i,k,t]}$$

$$\phi_i = \text{sigmoid}(\mu_1 + a_{id}[i, 1])$$

$$\mu \sim \mathcal{N}(\mathbf{0}_4,\, 1)$$
$$\sigma \sim \text{Exponential}(3)$$
$$\mathbf{L}_\rho \sim \text{LKJCholesky}(4,\, \eta = 3)$$

$$\mathbf{Z} \sim \mathcal{N}(\mathbf{0}_{4\times J},\, 1)$$
$$\mathbf{a}_{id} = \left(\mathbf{L}_\rho \cdot \text{diag}(\sigma) \cdot \mathbf{Z}\right)^\top$$

Where:

- $A_{[i,k,t]}$ is the attraction score of individual i for technique k at time t.
- $p_{[i,k,t]}$ is the personal yield (payoff) individual i obtained from technique k at time t — concretely, a measure of food-extraction efficiency (e.g. rate of fruit opening).
- $\phi_i \in [0, 1]$ is individual i's attraction updating weight. Near 1: attractions closely track the most recent payoff. Near 0: strong memory of cumulative past experience. It is constructed from a multivariate hierarchical prior shared by all four individual-level learning parameters (ϕ_i, γ_i, f_i, β_i)
- $\mu \in \mathbb{R}^4$ are population-level means
- $a_{id}[i, \cdot]$ are individual-level correlated deviations reconstructed via non-centered parameterization
- $\sigma \in \mathbb{R}^4_+$ are the per-parameter standard deviations (shrinkage parameters)
- $\mathbf{L}_\rho$ is the Cholesky factor of the 4×4 correlation matrix: the LKJ prior ($\eta = 3$) moderately shrinks correlations toward zero.
- $\mathbf{Z} \in \mathbb{R}^{J\times4}$ is a matrix of standard normal deviates (one row per individual).

3. Social Component — $S_{[i,k,t]}$

The social probability combines payoff bias with frequency-dependent conformity:

$$S_{[i,k,t]} = \frac{N_{[i,k,t]}^{f_i} \exp(\beta_i\, \pi_{[i,k,t]})}{\sum_{k'=1}^{K} N_{[i,k',t]}^{f_i} \exp(\beta_i\, \pi_{[i,k',t]})}$$
$$f_i = \exp(\mu_3 + a_{id}[i, 3])$$
$$\beta_i = \mu_4 + a_{id}[i, 4]$$

Where:

- $N_{[i,k,t]}$ is the *frequency-bias cue*: the number of times individual i observed conspecifics performing technique k at time t.

- $f_i > 0$ is individual i's conformity exponent. $f_i > 1$: positive frequency-dependent bias (disproportionate copying of the most common technique); $f_i = 1$: purely linear frequency weighting; $f_i < 1$: tendency to avoid the majority. The exponential link ensures $f_i > 0$; the prior $\mu_3 \sim \mathcal{N}(0, 1)$ corresponds to $f \approx 1$ a priori.
- $N^{f_i}_{[i,k',t]}$ in the denominator ranges over all K techniques with the same conformity exponent f_i, normalising the social probabilities.
- $\pi_{[i,k,t]}$ is the payoff observed from social demonstrators using technique k at time t.
- β_i is individual i's payoff-bias coefficient: positive values mean the individual preferentially copies techniques demonstrated with higher payoff. It is on the unconstrained real scale; the prior $\mu_4 \sim \mathcal{N}(0, 1)$ expresses prior uncertainty about the direction of payoff bias.

The numerator scores technique k by two multiplicative components: how often it was observed being used by conspecifics ($N^{f_i}_{[i,k,t]}$, frequency bias) and how profitable it appeared in their hands ($\exp(\beta_i\, \pi_{[i,k,t]})$, payoff bias). The denominator ($k' = 1, \ldots, K$) sums these scores over all techniques to normalise to a probability. Together they implement the idea that individuals copy techniques that are simultaneously common *and* profitable.

---

2. Social Learning (EWA-ILV)

The "EWA-ILV" model expands the EWA version by incorporating a wider array of social biases (kinship, prestige, age similarity, etc.) and modeling the linear influence of individual age on the learning parameters ϕ and γ.

Example

Python

```
from BayesForge import bf
import jax.numpy as jnp
import jax

m = bf(platform='cpu')

# Same data as Example 1 (d, cue, K, y_prev), plus the other social cues and age
m.data_on_model = dict(
    K=K,
    J=int(d["mono_index"].nunique()),
```

```
    tech=jnp.array(d["tech_index"].values - 1),
    id=jnp.array(d["mono_index"].values - 1),
    bout=jnp.array(d["forg_bout"].values),
    y_prev=y_prev,
    s=cue("s"),                        # frequency cue
    ps=cue("ps", scale=True),          # payoff cue
    ks=cue("ks", scale=True),          # kin cue
    pr=cue("prs", scale=True),         # rank (prestige) cue
    co=cue("cos", scale=True),         # cohort cue
    yo=cue("Yobs", scale=True),        # age (year of birth) cue
    age=jnp.array(d["age.c"].values), # centred age
)

def model(K, J, tech, id, bout, y_prev, s, ps, ks, pr, co, yo, age):
    # Priors (8 varying effects)
    lam   = m.dist.exponential(name="lambda", rate=1.0)
    mu    = m.dist.normal(name="mu", loc=jnp.zeros(8), scale=1.0)
    sigma = m.dist.exponential(name="sigma", rate=jnp.full(8, 3.0))
    L_Rho = m.dist.lkj_cholesky(name="L_Rho", dimension=8, concentration=3.0)
    z     = m.dist.normal(name="z", loc=jnp.zeros((8, J)), scale=1.0)
    b_age = m.dist.normal(name="b_age", loc=jnp.zeros(2), scale=1.0)

    # Varying effects
    L_scaled = L_Rho * sigma[:, None]
    a_id     = (L_scaled @ z).T

    # Parameters per individual with age effects
    phi_arr   = jax.nn.sigmoid(mu[0] + a_id[id, 0] + b_age[0] * age)
    gamma_arr = jax.nn.sigmoid(mu[1] + a_id[id, 1] + b_age[1] * age)
    fconf_arr = jnp.exp(mu[2] + a_id[id, 2])
    B_pay     = mu[3] + a_id[id, 3]
    B_kin     = mu[4] + a_id[id, 4]
    B_pres    = mu[5] + a_id[id, 5]
    B_coho    = mu[6] + a_id[id, 6]
    B_yob     = mu[7] + a_id[id, 7]

    # Scan carry: Attractions
    def step_fn(carry, x):
        AC = carry
        (phi_i, gamma_i, fconf_i, Bp, Bk, Bpr, Bc, By, t_i, id_i, b_i, py_i, s_i, ps_i, ks_i

        ac_new = jnp.where(b_i == 1, jnp.zeros(K), (1.0 - phi_i) * AC[id_i] + phi_i * py_i)
```

```
        # Asocial choice probability
        prob_individual = jax.nn.softmax(lam * ac_new)

        # Multi-cue social influence
        lin_rest = jnp.exp(Bp * ps_i[1:] + Bk * ks_i[1:] + Bpr * pr_i[1:] + Bc * co_i[1:] + B
        lin_mod = jnp.concatenate([jnp.ones(1), lin_rest]) * jnp.power(s_i, fconf_i)
        safe_denom = jnp.where(jnp.sum(lin_mod) > 0, jnp.sum(lin_mod), 1.0)
        prob_social = lin_mod / safe_denom

        # Mixture probability
        prob_mix = (1.0 - gamma_i) * prob_individual + gamma_i * prob_social
        has_social_info = jnp.logical_and(b_i > 1, jnp.sum(s_i) > 0)
        prob_final = jnp.where(has_social_info, prob_mix, prob_individual)

        return AC.at[id_i].set(ac_new), prob_final

    AC_init = jnp.zeros((J, K))
    xs = (phi_arr, gamma_arr, fconf_arr, B_pay, B_kin, B_pres, B_coho, B_yob,
          tech, id, bout, y_prev, s, ps, ks, pr, co, yo)
    _, probs_matrix = jax.lax.scan(step_fn, AC_init, xs)

    m.dist.categorical(name="obs", probs=probs_matrix, obs=tech)

m.fit(model, seed=0)
m.summary()
```

R

```
library(BayesForge)
m <- importBF(platform = "cpu")

# Same data as Example 1 (d, cue, K, y_prev), plus the other social cues and age
m$data_on_model <- list(
  K = K,
  J = length(unique(d$mono_index)),
  tech = jnp$array(as.integer(d$tech_index - 1)),
  id = jnp$array(as.integer(d$mono_index - 1)),
  bout = jnp$array(as.integer(d$forg_bout)),
  y_prev = jnp$array(y_prev),
  s = cue("s"),                    # frequency cue
```

```
  ps = cue("ps", scale = TRUE),    # payoff cue
  ks = cue("ks", scale = TRUE),    # kin cue
  pr = cue("prs", scale = TRUE),   # rank (prestige) cue
  co = cue("cos", scale = TRUE),   # cohort cue
  yo = cue("Yobs", scale = TRUE),  # age (year of birth) cue
  age = jnp$array(d$age.c)         # centred age
)

model <- function(K, J, tech, id, bout, y_prev, s, ps, ks, pr, co, yo, age) {
  # Priors (8 varying effects)
  lam   <- m$dist$exponential(name = "lambda", rate = 1.0)
  mu    <- m$dist$normal(name = "mu", loc = jnp$zeros(8L), scale = 1.0)
  sigma <- m$dist$exponential(name = "sigma", rate = jnp$full(8L, 3.0))
  L_Rho <- m$dist$lkj_cholesky(name = "L_Rho", dimension = 8L, concentration = 3.0)
  z     <- m$dist$normal(name = "z", loc = jnp$zeros(reticulate::tuple(8L, J)), scale = 1.0)
  b_age <- m$dist$normal(name = "b_age", loc = jnp$zeros(2L), scale = 1.0)

  # Varying effects
  L_scaled <- L_Rho * jnp$expand_dims(sigma, -1L)
  a_id     <- jnp$transpose(jnp$matmul(L_scaled, z))

  # Parameters per individual with age effects
  phi_arr   <- jax$nn$sigmoid(mu[0L] + a_id[id, 0L] + b_age[0L] * age)
  gamma_arr <- jax$nn$sigmoid(mu[1L] + a_id[id, 1L] + b_age[1L] * age)
  fconf_arr <- jnp$exp(mu[2L] + a_id[id, 2L])
  B_pay     <- mu[3L] + a_id[id, 3L]
  B_kin     <- mu[4L] + a_id[id, 4L]
  B_pres    <- mu[5L] + a_id[id, 5L]
  B_coho    <- mu[6L] + a_id[id, 6L]
  B_yob     <- mu[7L] + a_id[id, 7L]

  # Scan carry: Attractions
  step_fn <- function(carry, x) {
    AC <- carry
    phi_i <- x[[1]]; gamma_i <- x[[2]]; fconf_i <- x[[3]]
    Bp <- x[[4]]; Bk <- x[[5]]; Bpr <- x[[6]]; Bc <- x[[7]]; By <- x[[8]]
    t_i <- x[[9]]; id_i <- x[[10]]; b_i <- x[[11]]; py_i <- x[[12]]
    s_i <- x[[13]]; ps_i <- x[[14]]; ks_i <- x[[15]]; pr_i <- x[[16]]; co_i <- x[[17]]; yo_i

    ac_new <- jnp$where(b_i == 1L, jnp$zeros(K), (1.0 - phi_i) * AC[id_i] + phi_i * py_i)

    # Asocial choice probability
```

```
    prob_individual <- jax$nn$softmax(lam * ac_new)

    # Multi-cue social influence
    lin_rest <- jnp$exp(Bp * rest(ps_i) + Bk * rest(ks_i) + Bpr * rest(pr_i) + Bc * rest(co_
    lin_mod <- jnp$concatenate(list(jnp$ones(1L), lin_rest)) * jnp$power(s_i, fconf_i)
    safe_denom <- jnp$where(jnp$sum(lin_mod) > 0, jnp$sum(lin_mod), 1.0)
    prob_social <- lin_mod / safe_denom

    # Mixture probability
    prob_mix <- (1.0 - gamma_i) * prob_individual + gamma_i * prob_social
    has_social_info <- jnp$logical_and(b_i > 1L, jnp$sum(s_i) > 0)
    prob_final <- jnp$where(has_social_info, prob_mix, prob_individual)

    reticulate::tuple(AC$at[id_i]$set(ac_new), prob_final)
  }

  AC_init <- jnp$zeros(reticulate::tuple(J, K))
  xs <- reticulate::tuple(phi_arr, gamma_arr, fconf_arr, B_pay, B_kin, B_pres, B_coho, B_yob
                          tech, id, bout, y_prev, s, ps, ks, pr, co, yo)
  probs_matrix <- jax$lax$scan(step_fn, AC_init, xs)[[2]]

  m$dist$categorical(name = "obs", probs = probs_matrix, obs = tech)
}

m$fit(model, seed = 0L)
m$summary()
```

Julia

```
using PyBayesForge
using PythonCall
m = importBF(platform="cpu")

# Same data as Example 1 (d, cue, K, y_prev), plus the other social cues and age
m.data_on_model = pydict(
    K = K,
    J = length(unique(d.mono_index)),
    tech = jnp.array(d.tech_index .- 1),
    id = jnp.array(d.mono_index .- 1),
    bout = jnp.array(d.forg_bout),
    y_prev = jnp.array(y_prev),
```

```
    s = cue("s"),                    # frequency cue
    ps = cue("ps", scale=true),      # payoff cue
    ks = cue("ks", scale=true),      # kin cue
    pr = cue("prs", scale=true),     # rank (prestige) cue
    co = cue("cos", scale=true),     # cohort cue
    yo = cue("Yobs", scale=true),    # age (year of birth) cue
    age = jnp.array(d[:, "age.c"]),  # centred age
)

@BF function model(K, J, tech, id, bout, y_prev, s, ps, ks, pr, co, yo, age)
    # Priors (8 varying effects)
    lam   = m.dist.exponential(name="lambda", rate=1.0)
    mu    = m.dist.normal(name="mu", loc=jnp.zeros(8), scale=1.0)
    sigma = m.dist.exponential(name="sigma", rate=jnp.full(8, 3.0))
    L_Rho = m.dist.lkj_cholesky(name="L_Rho", dimension=8, concentration=3.0)
    z     = m.dist.normal(name="z", loc=jnp.zeros((8, J)), scale=1.0)
    b_age = m.dist.normal(name="b_age", loc=jnp.zeros(2), scale=1.0)

    # Varying effects
    L_scaled = L_Rho * jnp.expand_dims(sigma, -1)
    a_id     = jnp.transpose(jnp.matmul(L_scaled, z))

    # Parameters per individual with age effects
    phi_arr   = jax.nn.sigmoid(mu[0] + a_id[id, 0] + b_age[0] * age)
    gamma_arr = jax.nn.sigmoid(mu[1] + a_id[id, 1] + b_age[1] * age)
    fconf_arr = jnp.exp(mu[2] + a_id[id, 2])
    B_pay     = mu[3] + a_id[id, 3]
    B_kin     = mu[4] + a_id[id, 4]
    B_pres    = mu[5] + a_id[id, 5]
    B_coho    = mu[6] + a_id[id, 6]
    B_yob     = mu[7] + a_id[id, 7]

    # Scan carry: Attractions
    function step_fn(AC, x)
        AC = Py(AC)
        (phi_i, gamma_i, fconf_i, Bp, Bk, Bpr, Bc, By, t_i, id_i, b_i, py_i,
         s_i, ps_i, ks_i, pr_i, co_i, yo_i) = map(Py, x)

        ac_new = jnp.where(b_i == 1, jnp.zeros(K), (1.0 - phi_i) * AC[id_i] + phi_i * py_i)

        # Asocial choice probability
        prob_individual = jax.nn.softmax(lam * ac_new)
```

```
        # Multi-cue social influence
        lin_rest = jnp.exp(Bp * rest(ps_i) + Bk * rest(ks_i) + Bpr * rest(pr_i) + Bc * rest(
        lin_mod = jnp.concatenate(pylist([jnp.ones(1), lin_rest])) * jnp.power(s_i, fconf_i)
        safe_denom = jnp.where(jnp.sum(lin_mod) > 0, jnp.sum(lin_mod), 1.0)
        prob_social = lin_mod / safe_denom

        # Mixture probability
        prob_mix = (1.0 - gamma_i) * prob_individual + gamma_i * prob_social
        has_social_info = jnp.logical_and(b_i > 1, jnp.sum(s_i) > 0)
        prob_final = jnp.where(has_social_info, prob_mix, prob_individual)

        return (AC.at[id_i].set(ac_new), prob_final)
    end

    AC_init = jnp.zeros((J, K))
    xs = (phi_arr, gamma_arr, fconf_arr, B_pay, B_kin, B_pres, B_coho, B_yob,
          tech, id, bout, y_prev, s, ps, ks, pr, co, yo)
    probs_matrix = jax.lax.scan(step_fn, AC_init, xs)[1]

    m.dist.categorical(name="obs", probs=probs_matrix, obs=tech)
end

m.fit(model, seed=0)
m.summary()
```

Mathematical Details

The "Experience-Weighted Attraction with Individual Level Covariates" (EWA-ILV) model extends the EWA model in two ways: it incorporates a wider array of social cues in the social learning component, and it allows an individual's age to shift key learning parameters.

1. Mixture Likelihood

The observation likelihood has the same mixture form as the EWA model, using a three-part index: individual i, technique k, and time step t:

$$\theta_{[i,k,t]} = (1 - \gamma_i)\, I_{[i,k,t]} + \gamma_i\, S_{[i,k,t]}$$

2. Social Component — $S_{[i,k,t]}$

The social probability combines frequency-dependent conformity with a multi-cue log-linear model:

$$S_{[i,k,t]} = \frac{N_{[i,k,t]}^{f_i} \exp(B_{[i,k,t]})}{\sum_{k'=1}^{K} N_{[i,k',t]}^{f_i} \exp(B_{[i,k',t]})}$$

Where $B_{[i,1,t]} = 0$ by convention (technique 1 is the aliased baseline), and for $k > 1$:

$$B_{[i,k,t]} = \beta_{\text{pay},i}\, \pi_{[i,k,t]} + \beta_{\text{kin},i}\, \kappa_{[i,k,t]} + \beta_{\text{rank},i}\, \zeta_{[i,k,t]} + \beta_{\text{cohort},i}\, \eta_{[i,k,t]} + \beta_{\text{age},i}\, \alpha_{[i,k,t]}$$

3. Individual-Level Parameters, Age Effects, and Priors

All eight learning parameters are drawn from a correlated multivariate hierarchy. For each individual i, the varying parameters and deviations are constructed as:

$$\phi_i = \text{sigmoid}(\mu_0 + a_{id}[i,0] + \beta_\phi \cdot \text{age}_i)$$

$$\gamma_i = \text{sigmoid}(\mu_1 + a_{id}[i,1] + \beta_\gamma \cdot \text{age}_i)$$

$$f_i = \exp(\mu_2 + a_{id}[i,2])$$

$$\beta_{\text{pay},i} = \mu_3 + a_{id}[i,3]$$

$$\beta_{\text{kin},i} = \mu_4 + a_{id}[i,4]$$

$$\beta_{\text{rank},i} = \mu_5 + a_{id}[i,5]$$

$$\beta_{\text{cohort},i} = \mu_6 + a_{id}[i,6]$$

$$\beta_{\text{age},i} = \mu_7 + a_{id}[i,7]$$

$$\mathbf{a}_{id} = (\mathbf{L}_\rho \,\text{diag}(\sigma)\, \mathbf{Z})^\top$$

$$\mu \sim \mathcal{N}(\mathbf{0}_8, 1)$$

$$\sigma \sim \text{Exponential}(\text{rate} = 3)$$

$$\mathbf{L}_\rho \sim \text{LKJCholesky}(8, \eta = 3)$$

$$\mathbf{Z} \sim \mathcal{N}(\mathbf{0}_{8\times J}, 1)$$

$$\beta_\phi \sim \mathcal{N}(0,1), \quad \beta_\gamma \sim \mathcal{N}(0,1)$$

$$\lambda \sim \text{Exponential}(1)$$

Where:

- $N_{[i,k,t]}$ is the frequency cue: the number of times individual i observed conspecifics performing technique k at time t.
- $f_i > 0$ is individual i's conformity exponent, constructed with the exponential link to guarantee positive values. The population prior mean $\mu_2 \sim \mathcal{N}(0,1)$ corresponds to $f \approx 1$ (linear frequency copying) a priori.

- $\pi_{[i,k,t]}$ is the payoff observed from social demonstrators using technique k at time t. The prior $\mu_3 \sim \mathcal{N}(0,1)$ expresses uncertainty on payoff bias direction and magnitude.
- $\kappa_{[i,k,t]}$ is the kin bias cue: whether the demonstrators using technique k at time t are matrilineal kin of individual i. The coefficient $\beta_{\text{kin},i}$ models the kinship bias, with population prior mean $\mu_4 \sim \mathcal{N}(0,1)$.
- $\zeta_{[i,k,t]}$ is the rank/prestige cue: whether the demonstrators using technique k at time t are high-ranking (e.g., alpha) individuals. The coefficient $\beta_{\text{rank},i}$ models rank bias, with population prior mean $\mu_5 \sim \mathcal{N}(0,1)$.
- $\eta_{[i,k,t]}$ is the cohort cue: the age-similarity between individual i and demonstrators using technique k at time t. The coefficient $\beta_{\text{cohort},i}$ models cohort bias, with population prior mean $\mu_6 \sim \mathcal{N}(0,1)$.
- $\alpha_{[i,k,t]}$ is the age-bias cue: a year-of-birth prestige bias towards older/more experienced demonstrators using technique k at time t. The coefficient $\beta_{\text{age},i}$ models age-based prestige bias, with population prior mean $\mu_7 \sim \mathcal{N}(0,1)$.
- $\phi_i \in [0,1]$ and $\gamma_i \in [0,1]$ are individual-level attraction-updating and social-learning weights, constructed with the logistic/sigmoid link.
- age_i is the centered age of individual i.
- β_ϕ and β_γ are the population-level age slopes on the log-odds scale, determining how an individual's age alters their attraction updating and social learning weight.
- $\mathbf{a}_{id} \in \mathbb{R}^{J\times 8}$ is the matrix of individual-level deviations across the 8 varying parameters, reconstructed using the non-centered parameterization to optimize Hamiltonian Monte Carlo performance.
- $\mathbf{L}_\rho \in \mathbb{R}^{8\times 8}$ is the Cholesky factor of the correlation matrix, mapping correlations among all 8 parameters.
- $\sigma \in \mathbb{R}^8_+$ represents the standard deviations (shrinkage parameters) for each of the 8 parameters.
- $\mathbf{Z} \in \mathbb{R}^{8\times J}$ is a matrix of standard normal deviates.
- $\lambda > 0$ is the multinomial sensitivity parameter, governing choice deterministic behavior.

Model validation tests

- Comparison with Stan: EWA learning model on the panama_ewa data fitted with Stan and BayesForge, posteriors compared.
- Documentation examples: the Python, R and Julia examples of this page are run by the documentation tests (`Python/test_model_pages.py`, `R/test_r_model_pages.py`, `Julia/test_julia_model_pages.py`).

Notes

> **Note**
>
> - The `EWA-ILV` model is significantly more complex and requires larger sample sizes to resolve the correlations between varying effects.
> - Cholesky decomposition of the correlation matrix is used for both models to ensure positive-definiteness.

Reference(s)

Barrett, Brendan J., Richard L. McElreath, and Susan E. Perry. 2017. "Pay-Off-Biased Social Learning Underlies the Diffusion of Novel Extractive Foraging Traditions in a Wild Primate." *Proceedings of the Royal Society B: Biological Sciences* 284 (1856): 20170358. https://doi.org/10.1098/rspb.2017.0358.